

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ *Bruston*
Nom d'usage ▶ *Bruston Brouant*
Prénom ▶ *Thomas*
Adresse ▶ *780, rue des bouisses 34070 Montpellier.*

Titre professionnel visé

Développeur web et web mobile (niveau 5) RNCP 37674

MODALITÉ D'ACCÈS :

DOSSIER PROFESSIONNEL ^(DP)

☐ **X** Parcours de formation

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.
Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité	p.	8
CP1 Maquetter une application.....	p.	8
CP2 Interface statique	p.	14
CP3 Interface dynamique	p.	23

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité.	p.	33
CP5 et CP6 créer une base de données et les composants d'accès aux données	p.	33
CP7 Développer la partie back-end d'une application web ou web mobile.	p.	35
CP8 Documenter le déploiement	p.	39

Intitulé de l'activité-type n° 3	p.	
► Intitulé de l'exemple n° 1	p.	
p.		
► Intitulé de l'exemple n° 2	p.	
p.		
► Intitulé de l'exemple n° 3	p.	
p		

Intitulé de l'activité-type n° 4	p.	
► Intitulé de l'exemple n° 1	p.	
p.		
► Intitulé de l'exemple n° 2	p.	
p.		
► Intitulé de l'exemple n° 3	p.	
p		

Titres, diplômes, CQP, attestations de formation (facultatif)	p.	
Déclaration sur l'honneur	p.	
Documents illustrant la pratique professionnelle (facultatif)	p.	
Annexes (Si le RC le prévoit)	p.	

EXEMPLES DE PRATIQUE PROFESSIONNELLE



Activité-type 1

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité

Exemple n°1 ► CP1 Maquetter une application

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de ma formation DWWM, j'ai conçu et développé seul le site web d'un restaurant gastronomique fictif, Le Quai Antique. Avant toute phase de développement, j'ai réalisé une phase de conception complète incluant la rédaction des users stories et la création de maquettes fonctionnelles (wireframes) puis de maquettes graphiques (mockups) pour les formats mobile et desktop

En tant que	Je souhaite	Afin de
Visiteur	Voir la page d'accueil du restaurant	Découvrir l'établissement
Visiteur	Consulter les menus proposés	Choisir ce qui correspond à mes envies
Visiteur	Créer un compte	Pouvoir effectuer une réservation en ligne
Utilisateur connecté	Me connecter à mon espace personnel	Accéder au formulaire de réservation
Utilisateur connecté	Réserver une table en ligne	Planifier ma sortie au restaurant
Utilisateur connecté	Choisir le nombre de couverts, la date, l'heure et déclarer mes allergies	Personnaliser et sécuriser ma réservation
Administrateur	Me connecter à l'espace d'administration	Gérer les réservations et les menus

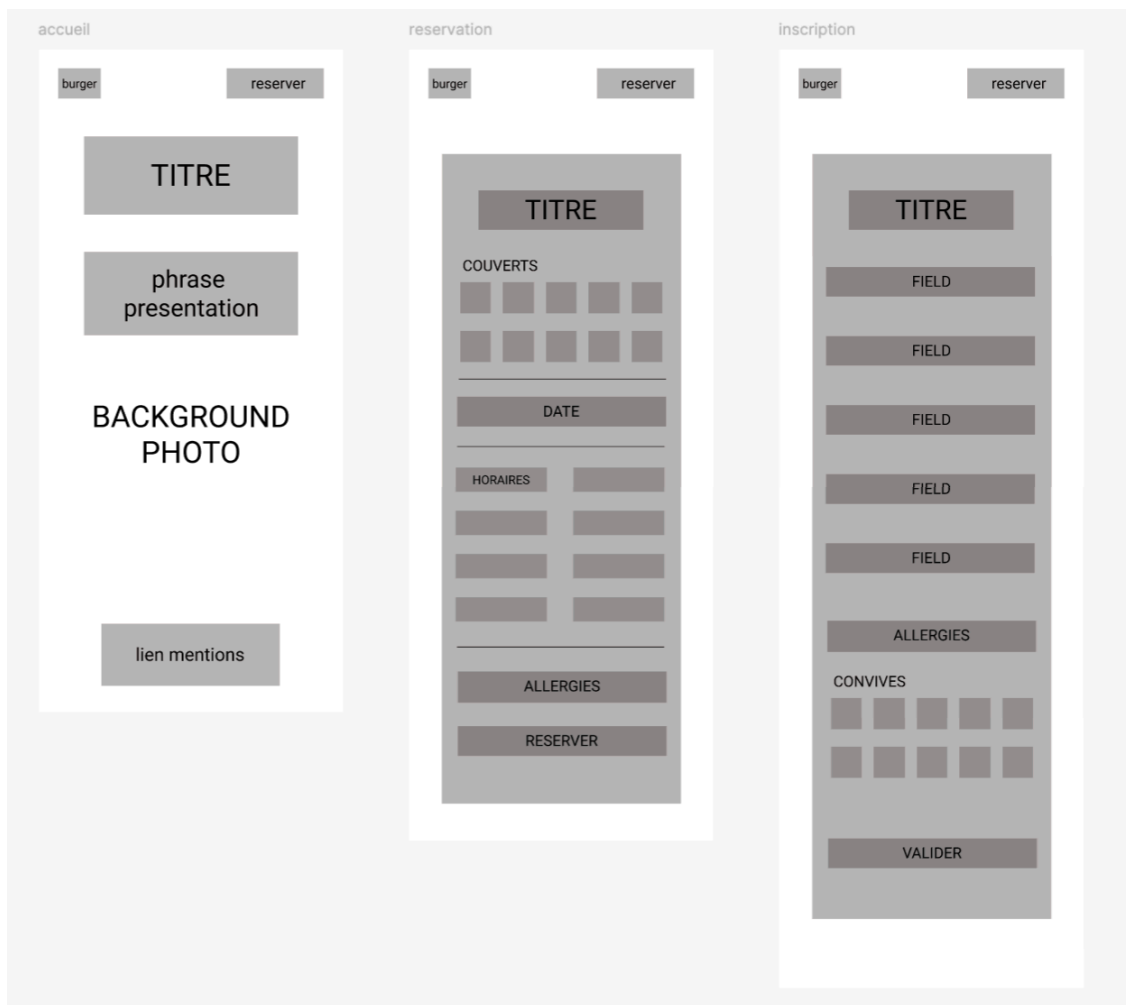
Diagramme de cas d'utilisation present en annexe (page 49)

2. Wireframes

À partir de ces users stories, j'ai réalisé des maquettes fonctionnelles (wireframes) pour les trois pages principales : accueil, réservation et inscription. Ces wireframes ont été produits en format mobile et desktop, en adoptant une approche mobile-first : la structure mobile est définie en premier, puis déclinée pour les écrans plus larges.

Les wireframes sont volontairement dépourvus de couleurs et de détails visuels — ils servent uniquement à valider la structure et la hiérarchie du contenu avant de passer au design.

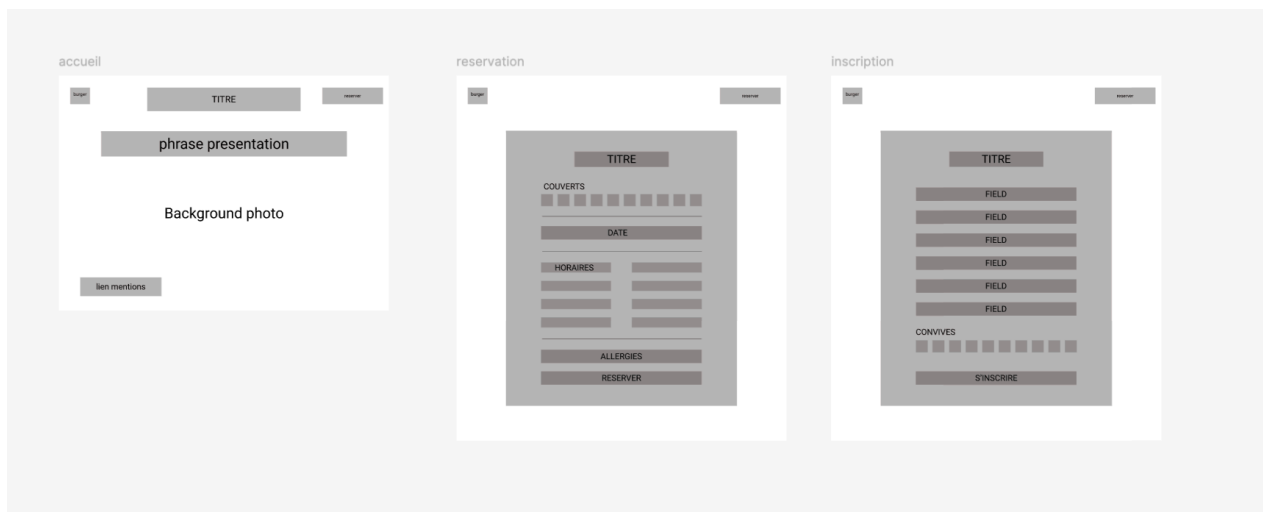
Wireframes format mobile : inscription, réservation, accueil



On peut observer sur ces wireframes mobiles :

- Un **header épuré** avec icône burger à gauche et bouton "Réserver" à droite, commun à toutes les pages
- La **page d'accueil** présente le titre du restaurant, une phrase de présentation, une photo en arrière-plan et un lien mentions légales en bas de page
- La **page de réservation** structure le formulaire en sections distinctes : sélection du nombre de couverts par cases cliquables (1 à 10), champ date, liste des horaires disponibles sur deux colonnes avec indicateur de disponibilité, champ allergies en texte libre, et bouton de validation
- La **page d'inscription** empile verticalement les champs nom, prénom, e-mail, mot de passe, confirmation du mot de passe, allergies, sélection du nombre de convives (cases 1 à 10), et bouton de soumission.

Wireframes format desktop : accueil, réservation, inscription

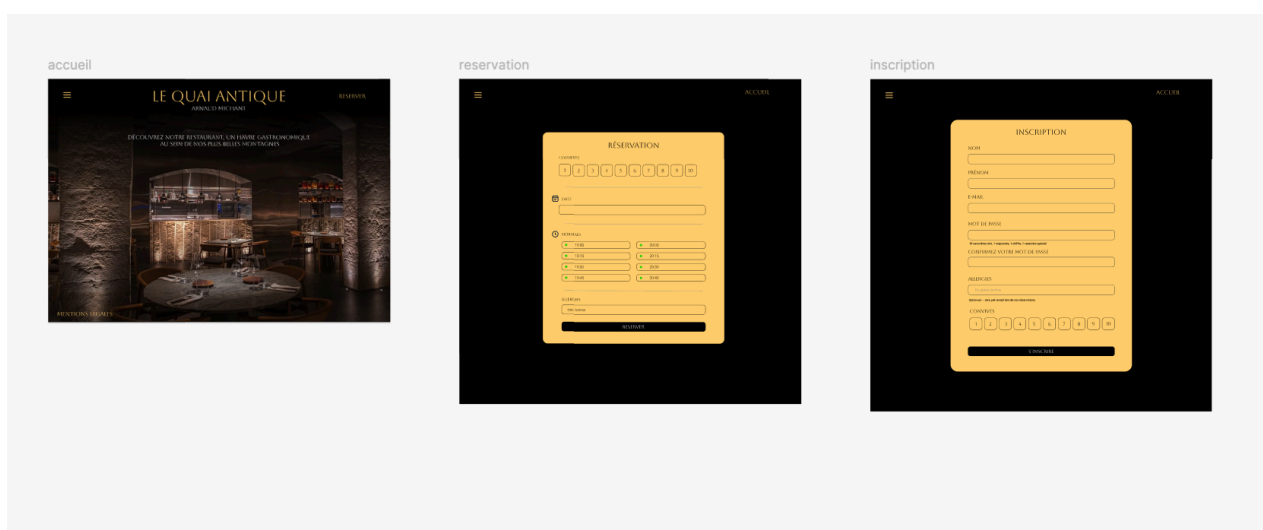


Sur les versions desktop, les éléments se réorganisent pour exploiter la largeur d'écran : le menu de navigation s'étend horizontalement, le contenu de l'accueil s'espace davantage, et les formulaires de réservation et d'inscription restent centrés avec leurs champs adaptés à la largeur disponible.

3. Mockups

À partir des wireframes validés, j'ai réalisé les maquettes graphiques (mockups) qui définissent l'identité visuelle finale du site. J'ai opté pour une charte sobre et haut de gamme : **fond noir**, **typographie et éléments d'interface dorés**, en cohérence avec l'univers de la gastronomie et l'identité du restaurant Le Quai Antique.

Mockups format desktop : accueil, réservation, inscription



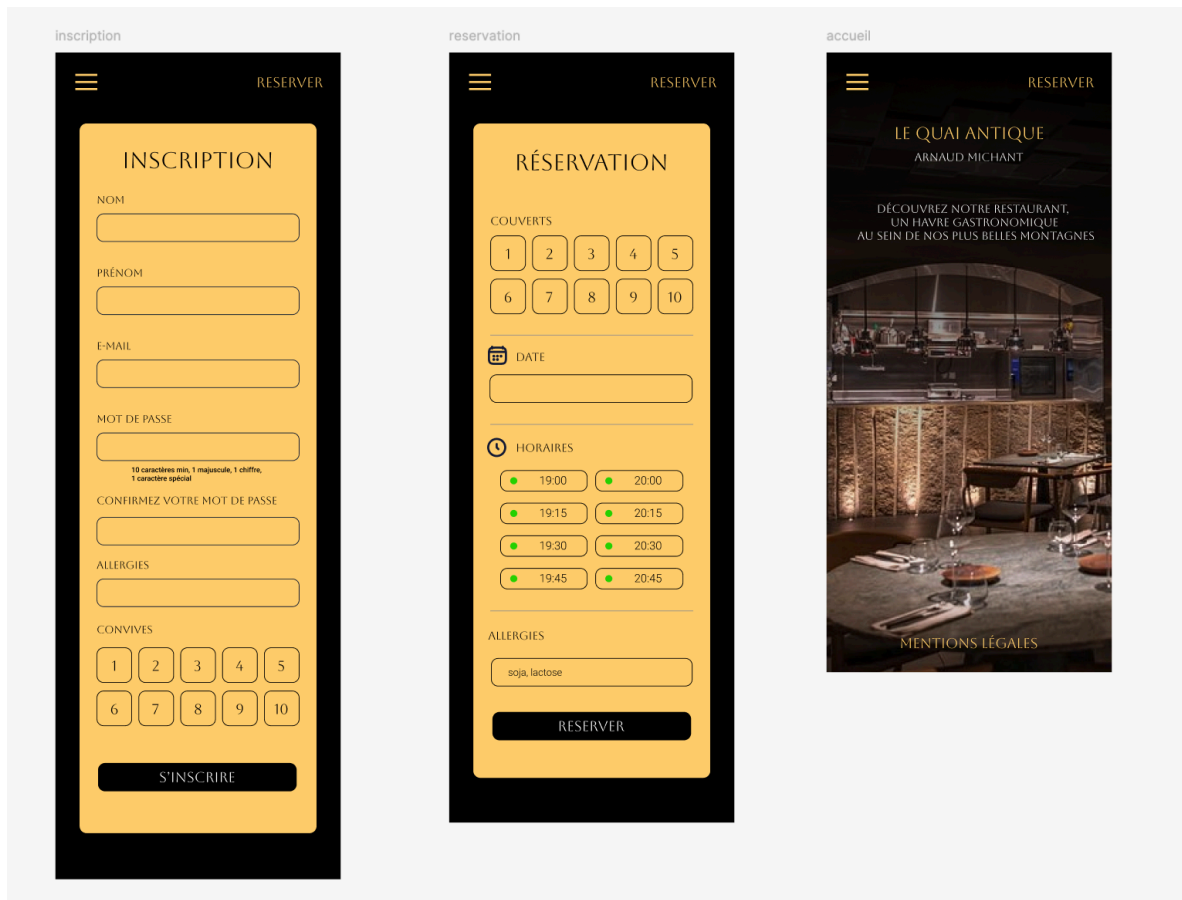
La **page d'accueil** exploite une photo immersive du restaurant en pleine largeur, avec le nom de l'établissement et une accroche en typographie serif dorée sur fond sombre

La **page de réservation** présente le formulaire dans un encart doré centré, les horaires disponibles signalés par des indicateurs verts, et un bouton de validation noir

La **page d'inscription** adopte le même parti-pris visuel avec l'encart doré, les champs clairement libellés en majuscules, et le message de règles du mot de passe affiché sous le champ concerné

Des captures d'écran sont disponibles en plus grand format en annexe.

Mockups format mobile : inscription, réservation, accueil



Sur mobile, le design conserve la même identité visuelle. La navigation se réduit au menu burger et au bouton "Réserver". Les formulaires occupent toute la largeur disponible, rendant la saisie confortable sur écran tactile. La sélection des couverts et des convives par cases numérotées est particulièrement adaptée à une interaction tactile.

Des captures d'écran sont disponibles en plus grand format en annexe.

2. Précisez les moyens utilisés :

Les wireframes et mockups ont été réalisés avec l'outil **Figma** qui permet de concevoir des maquettes multi-formats et de gérer les versions mobile et desktop au sein d'un même projet.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet, dans le cadre de ma formation au titre professionnel DWWM.

4. Contexte

Nom de l'entreprise, organisme ou association *Centre de formation STUDI*

Chantier, atelier, service ► *Projet fil rouge — CCP1 Front-end*

Période d'exercice ► Du : *20/02/2026* au : *01/03/2026*

5. Informations complémentaires (facultatif)

Des captures d'écran sont disponibles en plus grand format en annexe.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

Exemple n° 2 ► CP2 — Interface statique

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

En m'appuyant sur les maquettes réalisées lors de la CP1, j'ai développé seul l'intégralité des interfaces du site Le Quai Antique en HTML5, CSS3 vanilla et PHP, sans framework CSS. J'ai adopté une approche mobile-first et veillé à respecter les bonnes pratiques d'accessibilité RGAA et de référencement naturel (SEO).

1. Structure sémantique HTML5

Pour favoriser l'accessibilité et le référencement, j'ai structuré chaque page en utilisant les balises sémantiques HTML5 appropriées. Voici la structure générale du layout (base.php) :

```
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="<?= htmlspecialchars($metaDescription ?? 'Le Quai Antique
: Restaurant gastronomique du Chef Arnaud Michant.') ?>">
<title><?= htmlspecialchars($pageTitle ?? 'Le Quai Antique : Arnaud Michant') ?></title>
</head>
<body>
<header role="banner"> ... </header>
<main id="main-content"> <?= $content ?> </main>
<footer> ... </footer>
</body>
</html>
```

lang="fr" sur <html> : indispensable pour les lecteurs d'écran et le SEO

<meta name="description"> dynamique par page : optimisation SEO

<meta name="viewport"> : adaptation aux écrans mobiles

<main id="main-content"> : point d'ancrage pour la navigation rapide au clavier

htmlspecialchars() sur toutes les variables dynamiques : protection anti-XSS

JS chargé en fin de <body> : optimisation des performances de chargement

2. Navigation accessible

Le header intègre une navigation entièrement accessible, avec un menu burger pour mobile et un overlay de navigation complet :

```
<header class="site-header" role="banner">

<button
class="menu-burger"
aria-label="Ouvrir le menu de navigation"
aria-expanded="false"
aria-controls="nav-overlay" >

<span class="burger-line"></span>
<span class="burger-line"></span>
<span class="burger-line"></span>
</button>

<a href="/" aria-label="Le Quai Antique — Accueil">
<span class="header-brand-name">LE QUAI ANTIQUE</span>
</a>

<nav class="header-right" aria-label="Navigation principale">
<a href="/reservation" class="header-nav-link">RÉSERVER</a>
</nav>
</header>

<div id="nav-overlay" role="dialog" aria-modal="true"
aria-label="Menu de navigation" hidden>
<nav aria-label="Menu principal">
<ul class="nav-overlay-list">
<li><a href="/">Accueil</a></li>
<li><a href="/carte">La Carte</a></li>
<li><a href="/reservation">Réserver</a></li>
...
</ul>
</nav>
</div>
```

role="banner" sur <header> : rôle ARIA explicite

aria-label sur le bouton burger : description pour les lecteurs d'écran

aria-expanded="false" mis à jour par JavaScript à l'ouverture du menu

aria-controls="nav-overlay" : lien explicite entre le bouton et sa cible

role="dialog" + aria-modal="true" sur l'overlay : comportement d'accessibilité

hidden natif sur l'overlay : masquage sémantique

Deux <nav> avec aria-label distincts pour les différencier

3. CSS responsive et mobile-first

Le CSS est organisé en fichiers séparés par responsabilité (main.css, layout.css, components.css, pages.css) et utilise un système de variables CSS pour garantir la cohérence visuelle :

```
var(--font-display)
var(--font-body)
var(--font-ui)
var(--gold)
var(--black)
var(--transition)
```

Le layout principal utilise CSS Grid et Flexbox selon les contextes :

```
/* Header en grid 3 colonnes — burger | marque | nav */
.site-header {
  display: grid;
  grid-template-columns: auto 1fr auto;
}
```

```
/* Hero en flexbox centré */
.hero {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
}
```

```
/* Galerie en grid 3 colonnes */
.gallery-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

```
/* Carte des menus en grid 3 colonnes */
.card-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```



4. Media queries responsive

Le breakpoint principal est fixé à 768px. En dessous de ce seuil, les grilles passent en colonne unique et le header se réorganise pour centrer la marque :

```
@media (max-width: 780px) {  
  
  /* Header : centrage de la marque sur mobile */  
  
  .site-header {  
    grid-template-columns: 1fr auto 1fr;  
    padding: 1.5rem 1.5rem 1rem; }  
  
  .header-brand-name {  
    font-size: 1.1rem;  
    letter-spacing: 0.12em;  
    text-align: center; }  
  
  /* Sous-titre masqué sur mobile */  
  
  .header-brand-sub { display: none; }  
  
  /* Galerie : 3 colonnes → 2 colonnes */  
  
  .gallery-grid {  
    grid-template-columns: repeat(2, 1fr);  
    padding: 0 1rem; }  
  
  /* Carte des menus : 3 colonnes → 1 colonne */  
  
  .card-grid {  
    grid-template-columns: 1fr;  
    gap: 3rem; }  
  
  /* Section carte : padding réduit */  
  
  .card-section { padding: 7rem 1.5rem 4rem; }  
  
  /* Footer : navigation centrée */  
  
  .footer-nav {  
    justify-content: center;  
    gap: 1.5rem; } }
```

5. Rendu final

Rendu desktop : page d'accueil



Sur desktop, le header affiche le nom du restaurant centré avec le sous-titre "ARNAUD MICHANT", le menu burger à gauche et le lien "RÉSERVER" à droite. La photo du restaurant occupe toute la hauteur de l'écran avec la phrase d'accroche centrée.

Rendu mobile : page d'accueil



LE QUAI ANTIQUE

RÉSERVER

DÉCOUVREZ NOTRE RESTAURANT, UN
HAVRE GASTRONOMIQUE
AU SEIN DE NOS PLUS BELLES
MONTAGNES.



MENTIONS LÉGALES

2. Précisez les moyens utilisés :

Éditeur de code : Visual Studio Code

Langages : HTML5, CSS3 vanilla, PHP 8

Organisation CSS : fichiers séparés par responsabilité — main.css, layout.css, components.css, pages.css

Typographies : Google Fonts (Cinzel, Cormorant Garamond, Jost)

Icônes : Font Awesome 6

Environnement : Docker (PHP + Nginx + MySQL)

Versionning : Git

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet, dans le cadre de ma formation au titre professionnel DWWM.

4. Contexte

Nom de l'entreprise, organisme ou association ► Centre de formation STUDI

Chantier, atelier, service ► Projet fil rouge : CCP1 Front-end

Période d'exercice ► Du : 20/02/2026 au : 01/03/2026

5. Informations complémentaires (facultatif)

Le projet est versionné sur GitHub. Le code source est disponible dans le dépôt Quai Antique.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

Exemple n° 3 ► CP3 Interface dynamique

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet Le Quai Antique, j'ai développé seul le formulaire de réservation en ligne. Ce formulaire intègre des **interactions dynamiques en JavaScript vanilla** sans rechargement de page : chargement des créneaux horaires disponibles via AJAX, sélection interactive du nombre de couverts, et activation conditionnelle du bouton de soumission.

1. Structure du formulaire

Le formulaire est structuré en plusieurs sections avec une attention particulière portée à l'accessibilité :

```

<form action="/reservation" method="POST" novalidate>

<!-- Token CSRF anti-falsification -->

<input type="hidden" name="csrf_token"
value="<?= htmlspecialchars($_SESSION['csrf_token'] ?? '') ?>">
<input type="hidden"
name="heure"
id="heure-hidden" value="">
<input type="hidden" name="nombre_convives" id="nombre_convives" value="1">

<!-- Sélecteur de couverts accessible -->

<div class="convives-selector"
role="group"
aria-label="Nombre de couverts">
<?php for ($i = 1; $i <= 10; $i++): ?>
<button type="button"
class="convives-btn <?= $i === 1 ? 'active' : '' ?>"
data-value="<?= $i ?>"
aria-label="<?= $i ?> couvert<?= $i > 1 ? 's' : '' ?>"
aria-pressed="<?= $i === 1 ? 'true' : 'false' ?>"> <?= $i ?>
</button>
<?php endfor; ?>
</div>

<!-- Sélecteur de date avec minimum = aujourd'hui -->

<input type="date"
id="date"
name="date"
min="<?= date('Y-m-d') ?>"
required aria-required="true">

<!-- Grille des créneaux — remplie dynamiquement par JS -->

<div id="slots-grid"
role="group"
aria-label="Créneaux horaires disponibles"
style="display: none; >>
</div>

<!-- Bouton désactivé tant qu'aucun créneau sélectionné -->
<button type="submit" id="btn-reserver" disabled>RÉSERVER</button>
</form>

```


Token CSRF sur tous les formulaires POST : protection anti-falsification de requête
role="group" + aria-label sur le sélecteur de couverts et la grille de créneaux
aria-pressed sur les boutons de sélection : pattern ARIA toggle button
min="<?= date('Y-m-d') ?>" : empêche la sélection d'une date passée côté client
Bouton de soumission désactivé par défaut : activé uniquement après sélection d'un créneau
htmlspecialchars() sur toutes les sorties : protection anti-XSS

2. Chargement dynamique des créneaux (AJAX)

Lorsque l'utilisateur sélectionne une date, un appel fetch() est déclenché vers l'endpoint /reservation/check-availability pour récupérer les créneaux disponibles sans rechargement de page :

```

document.addEventListener('DOMContentLoaded', () => {
  const dateInput = document.getElementById('date');
  const slotsGrid = document.getElementById('slots-grid');
  const slotsPlaceholder = document.getElementById('slots-placeholder');
  const heureHidden = document.getElementById('heure-hidden');
  const btnReserver = document.getElementById('btn-reserver');

  // Déclenchement au changement de date

  dateInput.addEventListener('change', () => {
    const date = dateInput.value;
    if (!date) return;

    // Réinitialisation de l'état

    slotsGrid.style.display = 'none';
    heureHidden.value = '';
    btnReserver.disabled = true;

    // Appel AJAX vers l'endpoint PHP

    fetch(`/reservation/check-availability?date=${encodeURIComponent(date)}`)
      .then(res => {
        if (!res.ok) throw new Error('Erreur réseau');
        return res.json(); })
      .then(data => {
        if (data.error) {
          slotsPlaceholder.textContent = data.error;
          slotsPlaceholder.style.display = 'block';
          return; }
        renderSlots(data.slots); })
      .catch(() => {
        slotsPlaceholder.textContent =
          'Impossible de charger les créneaux. Veuillez réessayer.';
        slotsPlaceholder.style.display = 'block'; }); });

```

encodeURIComponent() sur le paramètre date : protection contre les injections dans l'URL

Gestion complète des cas d'erreur : erreur réseau (.catch()), erreur métier (vérification data.error)

Réinitialisation systématique de l'état avant chaque nouvel appel

3. Rendu dynamique des créneaux

Les créneaux reçus en JSON sont transformés en boutons interactifs et injectés dans le DOM :

```
function renderSlots(slots) {
  slotsGrid.innerHTML = '';
  if (!slots || slots.length === 0) {
    slotsPlaceholder.textContent = 'Aucun créneau disponible pour cette date.';
    slotsPlaceholder.style.display = 'block';
    return; }

  slots.forEach(slot => {
    const btn = document.createElement('button');
    btn.type = 'button';
    btn.className = 'slot-btn' + (slot.available ? '' : ' unavailable');

    // Attributs ARIA pour l'accessibilité

    btn.setAttribute('aria-label', slot.heure + (slot.available ? ' disponible' : ' complet'));
    btn.setAttribute('aria-pressed', 'false');
    btn.innerHTML = `
    <span class="slot-dot ${slot.available ? '' : 'unavailable'}"
    aria-hidden="true">
    </span>
    ${slot.heure} `;
    if (slot.available) {
      btn.addEventListener('click', () => selectSlot(btn, slot.heure)); }
    slotsGrid.appendChild(btn); });
  slotsGrid.style.display = 'grid'; }
```

4. Sélection d'un créneau

La sélection d'un créneau met à jour l'état visuel des boutons, le champ caché heure puis active le bouton de soumission :

```

function selectSlot(btn, heure) {

    // Réinitialiser tous les boutons

    slotsGrid.querySelectorAll('.slot-btn').forEach(b => {
    b.classList.remove('active');
    b.setAttribute('aria-pressed', 'false'); });

    // Activer le bouton sélectionné

    btn.classList.add('active');
    btn.setAttribute('aria-pressed', 'true');

    // Transmettre la valeur au formulaire

    heureHidden.value = heure;
    btnReserver.disabled = false; }

```

5. Endpoint PHP : traitement de la requête AJAX

Côté serveur, le ReservationController traite la requête et retourne une réponse JSON :

```

public function checkAvailability(): void {
    header('Content-Type: application/json');
    $date = trim($this->get('date') ?? ''); /

    / Validation de la date
    if (empty($date)) {
        echo json_encode(['error' => 'Date manquante.']);
        exit; }

    // Vérification jour d'ouverture (fermé le lundi)

    if (!$this->reservationService->isValidDay($date)) {
        echo json_encode(['error' => 'Le restaurant est fermé le lundi.']);
        exit; }

    try {
        $heures = $this->reservationService->getAvailableSlots($date);
        $slots = [];
        foreach ($heures as $heure) {
            $slots[] = [
                'heure' => $heure,
                'available' => $this->reservationService->isAvailable($date, 1), ]; }
        echo json_encode(['slots' => $slots]); }
    catch (\RuntimeException $e) {
        echo json_encode(['error' => 'Erreur lors du chargement des créneaux.']); }
    exit; }

```

header('Content-Type: application/json') : réponse correctement typée

Validation de la date avant tout traitement

Logique métier déléguée au ReservationService : respect du principe de responsabilité unique

Gestion des exceptions avec try/catch : style défensif

6. Validation côté serveur

La soumission du formulaire est traitée par la méthode store() du contrôleur, qui vérifie le token CSRF, valide toutes les données et crée la réservation via le Repository :

```

if (!Session::isLoggedIn() ) {
    $this->redirect('/connexion'); }

// Vérification CSRF
$this->verifyCsrf();
$date = trim($this->post('date'));
$heure = trim($this->post('heure'));
$nbConvives = (int) $this->post('nombre_convives');
$allergies = trim($this->post('allergies')) ?: null;

// Validation métier

$errors = $this->reservationService->validate($date, $heure, $nbConvives);
if (!empty($errors)) {
    Session::setFlash('errors', $errors);
    $this->redirect('/reservation'); }
try { $reservation = new Reservation(
    userId: Session::getUserId(),
    date: $date,
    heure: $heure,
    nombreConvives: $nbConvives,
    allergies: $allergies );
    $this->reservationRepository->create($reservation);
    Session::setFlash('success', 'Votre réservation a bien été enregistrée. ');
    $this->redirect('/mon-compte/reservations'); }
catch (\RuntimeException $e) {
    Session::setFlash('errors', [$e->getMessage()]);
    $this->redirect('/reservation'); } }

```

2. Précisez les moyens utilisés :

JavaScript vanilla : fetch(), addEventListener(), createElement(), querySelectorAll()
 PHP 8 orienté objet : Controller, Service, Repository, Entity
 Architecture MVC maison : séparation stricte des responsabilités
 PDO avec requêtes préparées : accès sécurisé à la base de données
 IDE : Visual Studio Code

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul sur ce projet, dans le cadre de ma formation au titre professionnel DWWM.

4. Contexte

Nom de l'entreprise, organisme ou association *Centre de formation STUDI*

Chantier, atelier, service ▶ *Projet fil rouge — CCP1 Front-end*

Période d'exercice ▶ Du : *20/02/2026* au : *01/03/2026*

5. Informations complémentaires (facultatif)

Le projet est versionné sur GitHub. Le code source est disponible dans le dépôt du projet Quai Antique.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité.

Exemple n° 1 ▶ *CP5 et CP6 créer une base de données et les composants d'accès aux données*

Dans le cadre de ma formation DWWM, j'ai conçu et développé seul la base de données relationnelle de la plateforme de coaching sportif Ju Coach Sportif. Cette application permet à un coach de gérer ses clients, ses programmes sportifs, sa galerie photo et sa messagerie interne. J'ai suivi la méthode Merise en trois étapes : modèle conceptuel (MCD), modèle logique (MLD) puis modèle physique (MPD).

1. Modèle Conceptuel de Données (MCD)

Disponible en annexe

2. Modèle Logique de Données (MLD)

En appliquant les règles de passage du MCD au MLD, j'obtiens les tables suivantes :

ROLE (id, libelle)

UTILISATEUR (id, email, password, prenom, nom, telephone, adresse, statut, created_at, #role_id)

PASSWORD_RESET (id, token, expire_at, utilise, #utilisateur_id)

QUESTIONNAIRE (id, age, genre, taille, poids, objectif_principal, condition_physique, ..., #utilisateur_id)

PROGRAMME (id, type, titre, description, objectifs, inclut, tarifs, prix, created_at)

COMMANDE (id, date, montant, #utilisateur_id, #programme_id)

PLAN_NUTRITIONNEL (id, nom, date_creation)

UTILISATEUR_PLAN (utilisateur_id, plan_id, date_assignment, envoye)

GALERIE_PHOTO (id, fichier, legende, ordre, date_ajout)

GALERIE_VIDEO (id, url_youtube, titre, description, date_ajout)

CONTACT (id, nom, email, message, date_envoi, lu)

Règles appliquées :

- Les relations **1:N** sont matérialisées par une **clé étrangère** dans la table du côté "1" (ex: role_id dans utilisateur)
- La relation **N:N** entre utilisateur et plan_nutritionnel est matérialisée par la **table de liaison** utilisateur_plan avec ses deux clés étrangères en clé primaire composite

3. Modèle Physique de Données (MPD)

J'ai implémenté la base de données en MySQL 8 via un script SQL exécuté automatiquement au démarrage du container Docker. Voici un exemple du script :

```
CREATE TABLE IF NOT EXISTS utilisateur (  
  id      INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  email   VARCHAR(255) NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  prenom  VARCHAR(100) NOT NULL,  
  nom     VARCHAR(100) NOT NULL,  
  telephone VARCHAR(30) DEFAULT NULL,  
  adresse TEXT          DEFAULT NULL,
```



```

statut  ENUM('actif','inactif') NOT NULL DEFAULT 'actif',
role_id INT UNSIGNED NOT NULL,
created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (id),
UNIQUE KEY uq_utilisateur_email (email),
CONSTRAINT fk_utilisateur_role
    FOREIGN KEY (role_id) REFERENCES role (id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

ENGINE=InnoDB sur toutes les tables

DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci : encodage complet supportant tous les caractères

ON DELETE CASCADE sur les clés étrangères des tables enfants : suppression en cascade

ON DELETE RESTRICT sur les relations métier critiques : empêche la suppression d'un rôle utilisé

UNIQUE KEY sur les champs devant être uniques (email, token)

Champ ENUM pour les valeurs contraintes : validation au niveau de la base de données

Dans le cadre du projet Ju Coach Sportif, j'ai développé seul la couche d'accès aux données en PHP 8 orienté objet. Cette couche repose sur le pattern **Repository** : chaque entité de la base de données dispose d'une classe Repository dédiée qui centralise toutes les opérations SQL concernées. Tous les accès à la base de données passent exclusivement par **PDO avec requêtes préparées**.

4. Connexion PDO centralisée

La connexion à la base de données est gérée par une classe Database implémentant le pattern **Singleton** : une seule instance de connexion est créée pour toute la durée de la requête :

```

// Core/Database.php
$this->pdo = Database::getInstance()->getConnection();

```

Cette instance est injectée dans tous les repositories via la classe abstraite AbstractRepository :

```

abstract class AbstractRepository
{
    protected PDO $pdo;
}

```

```

public function __construct()
{
    $this->pdo = Database::getInstance()->getConnection();
}
}

```

abstract class : force l'héritage

protected PDO \$pdo : propriété accessible uniquement aux classes filles

Connexion centralisée : pas de duplication de code entre les repositories

Pattern Singleton sur Database : une seule connexion PDO par requête

5. Opérations CRUD avec requêtes préparées

Toutes les opérations SQL utilisent exclusivement des requêtes préparées PDO .

Voici le détail d'une opération : Lecture : findAll()

```

public function findAll(): array {
    try {
        $stmt = $this->pdo->prepare
        ( 'SELECT * FROM programme ORDER BY id ASC' );
        $stmt->execute();

        return array_map(fn($row) => Programme::fromArray($row), $stmt->fetchAll()); }
    catch (PDOException $e) {
        error_log('[ProgrammeRepository::findAll] ' . $e->getMessage());
        throw new \RuntimeException('Erreur de récupération des programmes.');
```

6. Points de sécurité et bonnes pratiques

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Sur l'ensemble de la couche d'accès aux données :

- **Requêtes préparées PDO exclusivement** : les paramètres sont liés via des placeholders nommés (:id, :titre...)
- **Typage strict** : declare(strict_types=1) en tête de chaque fichier, paramètres typés (int \$id, Programme \$programme) et retours typés (?Programme, bool, int)

- **Gestion des exceptions** : chaque méthode encapsule ses opérations dans un try/catch(PDOException \$e). L'exception PDO est loguée avec error_log() pour la traçabilité.
- **Hydratation via fromArray()** : les données brutes de la BDD sont toujours transformées en objets Programme typés avant d'être retournées, jamais exposées sous forme de tableaux non contrôlés

7. Accès aux données NoSQL : MongoDB

Dans le projet Ju Coach Sportif, j'ai intégré MongoDB comme base de données non relationnelle pour les statistiques de commandes de programmes. Ces données sont enregistrées dans une collection commandes_stats via l'extension PHP MongoDB :

```
class MongoService
{
    private Collection $collection;

    public function __construct()
    {
        $host    = Env::get('MONGO_HOST',    'mongo');
        $port    = Env::get('MONGO_PORT',    '27017');
        $user     = Env::get('MONGO_ROOT_USER', 'root');
        $password = Env::get('MONGO_ROOT_PASSWORD', 'rootpassword');
        $database = Env::get('MONGO_DATABASE', 'ju_coach_stats');

        $uri = "mongodb://{ $user }:{ $password }@{ $host }:{ $port }";

        $client    = new Client($uri);
        $this->collection = $client->$database->commandes_stats;
    }
}
```

```

public function enregistrerCommande(
    int $commandId,
    int $programmId,
    string $programmeTitre,
    string $programmeType,
    string $dateAchat,
    float $montant
): void {
    try {
        $this->collection->insertOne([
            'commande_id' => $commandId,
            'programme_id' => $programmId,
            'programme_titre' => $programmeTitre,
            'programme_type' => $programmeType,
            'date_achat' => $dateAchat,
            'montant' => $montant,
            'statut' => 'choisi',
            'created_at' => new \MongoDB\BSON\UTCDateTime(time() * 1000),
        ]);

    } catch (\Exception $e) {
        error_log('[MongoService::enregistrerCommande] ' . $e->getMessage());
    }
}

```

Ces données alimentent les statistiques de l'espace administrateur. MongoDB a été choisi car ces données agrégées ne nécessitent pas de contraintes relationnelles, une base orientée document est adaptée pour ce type d'informations.

Exemple n° 2 ► CP7 — Développer la partie back-end d'une application web ou web mobile.

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet Ju Coach Sportif, j'ai développé seul l'intégralité du back-end en PHP 8 orienté objet pur, sans framework. L'application repose sur une architecture MVC maison avec un Router centralisé, des Controllers, des Services et des Repositories. J'ai porté une attention particulière à la sécurité : contrôle d'accès par rôle, protection CSRF, hashage des mots de passe, tokens de réinitialisation, et style défensif sur toutes les entrées.

Le point d'entrée unique public/index.php instancie le Router et dispatch toutes les requêtes :

```
php
// public/index.php
$router = new Router();
$router->dispatch($_SERVER['REQUEST_METHOD'], $_SERVER['REQUEST_URI']);
```

1. Router avec contrôle d'accès par rôle

Le Router centralise toutes les routes de l'application et gère le contrôle d'accès. Chaque route est déclarée avec sa méthode HTTP, son chemin, son controller, son action et le rôle requis :

```
private function registerRoutes(): void
{
    // Route publique — aucun rôle requis
    $this->add('GET', '/programmes', 'ProgrammeController', 'index', null);

    // Route utilisateur connecté
    $this->add('GET', '/mon-espace', 'ProfilController', 'showProfil', 'utilisateur');

    // Route admin uniquement
    $this->add('GET', '/admin', 'AdminController', 'dashboard', 'administrateur');
    $this->add('POST', '/admin/programme/nouveau', 'ProgrammeController', 'create',
'administrateur');
}
```

Le contrôle d'accès repose sur une **hiérarchie de rôles** :

```
private function checkAccess(?string $requiredRole): void
{
    if ($requiredRole === null) return;

    // Redirection vers connexion si non authentifié
    if (!Session::isLoggedIn()) {
        Session::set('redirect_after_login', $_SERVER['REQUEST_URI']);
        header('Location: /connexion');
        exit;
    }

    // Hiérarchie des rôles : administrateur > utilisateur
    $hierarchy = ['utilisateur' => 1, 'administrateur' => 2];

    $userLevel = $hierarchy[Session::getUserRole() ] ?? 0;
    $requiredLevel = $hierarchy[$requiredRole] ?? 0;

    // Accès refusé si niveau insuffisant
    if ($userLevel < $requiredLevel) {
        $this->abort(403);
    }
}
```

3. AuthController — authentification complète

L'AuthController gère l'ensemble du cycle d'authentification : connexion, inscription, déconnexion, mot de passe oublié et réinitialisation.

Connexion :

```
public function login(): void
{
    // Vérification du token CSRF
    $this->verifyCsrf();

    $email = trim($this->post('email'));
    $password = $this->post('password');

    try {
        $user = $this->authService->login($email, $password);

        if ($user === null) {
            Session::setFlash('errors', ['Email ou mot de passe incorrect.']);
        }
    }
}
```

```

        $this->redirect('/connexion');
    }

    // Redirection selon le rôle
    $this->redirect($user->isAdmin() ? '/admin' : '/');

} catch (\RuntimeException $e) {
    Session::setFlash('errors', [$e->getMessage()]);
    $this->redirect('/connexion');
}
}

```

Mot de passe oublié avec token sécurisé :

```

public function forgot(): void
{
    $this->verifyCsrf();

    $email = trim($this->post('email'));

    try {
        $token = $this->authService->forgot($email);

        if ($token !== null) {
            $mailService = new MailService();
            $mailService->sendReinitialisationMdp($email, $token);
        }

        // Message générique — ne révèle pas si l'email existe (OWASP)
        Session::setFlash('success',
            'Si un compte correspond à cet email, un lien de réinitialisation a été envoyé.'
        );
        $this->redirect('/mot-de-passe-oublie');

    } catch (\InvalidArgumentException $e) {
        Session::setFlash('errors', [$e->getMessage()]);
        $this->redirect('/mot-de-passe-oublie'); }}

```

Le diagramme de sequence « commander un programme sportif » est disponible en annexe (page 50)

Mesures de sécurité

- **Injection SQL** : PDO avec requêtes préparées
- **XSS** : htmlspecialchars() sur toutes les sorties HTML
- **CSRF** : token vérifié sur tous les formulaires POST via verifyCsrf()
- **Mots de passe** : hashés avec password_hash()
- **Accès non autorisé** : contrôle d'accès par rôle sur chaque route du Router
- **Énumération d'emails** : message générique sur mot de passe oublié
- **Tokens de reset** : token unique 64 caractères, durée limitée, usage unique
- **Credentials** : variables d'environnement via .env, jamais committées

Dans le cadre du projet Vite & Gourmand, j'ai assuré seul le déploiement complet de l'application en production sur un VPS OVH (Ubuntu 22.04 LTS), en conteneurisant l'ensemble de la stack avec Docker Compose. N'ayant pas déployé les applications Quai Antique et Ju coach sportif j'ai décidé de vous présenter un exemple réel de déploiement appliqué au site Vite et Gourmand.

1. Choix de l'infrastructure

J'ai choisi un VPS OVH (VPS 1, Ubuntu 22.04 LTS, IP 57.129.41.2) pour sa compatibilité native avec Docker et sa capacité à héberger simultanément les quatre conteneurs nécessaires : PHP/ Nginx, MySQL 8, MongoDB 7 et Nginx reverse proxy.

2. Configuration du serveur et installation de Docker

Connexion SSH initiale puis installation de Docker et Docker Compose sur le serveur Ubuntu :

```
ssh ubuntu@57.129.41.2
sudo apt update && sudo apt upgrade -y
sudo apt install docker.io docker-compose-plugin -y
sudo usermod -aG docker ubuntu
```

3. Déploiement de l'application

Clonage du dépôt GitHub, configuration des variables d'environnement de production avec le fichier .env (jamais versionné), puis lancement avec le fichier dédié à la production : docker-compose.prod.yml

```
git clone https://github.com/[username]/viteGourmandV2.git
cd viteGourmandV2
cp .env.example .env
# Edition du .env avec les credentials de production
```

`docker compose -f docker-compose.prod.yml up -d --build`

Le `docker-compose.prod.yml` diffère de la version de développement sur plusieurs points : absence de PhpMyAdmin, volumes persistants pour MySQL et MongoDB, redémarrage automatique des conteneurs (`restart: always`), et variables d'environnement injectées depuis le fichier `.env`.

4. Nom de domaine et HTTPS

J'ai configuré le nom de domaine `vitegourmand.tech` en pointant les enregistrements DNS de type A vers l'IP du VPS dans l'interface OVH. Le certificat SSL Let's Encrypt a ensuite été généré via Certbot :

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d vitegourmand.tech -d www.vitegourmand.tech
```

Certbot a automatiquement modifié la configuration Nginx pour rediriger tout le trafic HTTP vers HTTPS.

Exemple n° 3 ► CP8 — Documenter le déploiement

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

5. Gestion des variables d'environnement et sécurité

Toutes les informations sensibles (credentials BDD, clé SMTP, secret de session) sont stockées dans le fichier `.env`, listé dans le `.gitignore` et jamais push sur GitHub.

VPS OVH Ubuntu 22.04 LTS , Docker + Docker Compose , Nginx , Certbot / Let's Encrypt , Git , GitHub , SSH , domaine `vitegourmand.tech`

J'ai travaillé seul sur ce projet, dans le cadre de ma formation au titre professionnel DWWM.

Titres, diplômes, CQP, attestations de formation

Déclaration sur l'honneur

Je soussigné(e) [prénom et nom] **Thomas Bruston**.....,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis
l'auteur(e) des réalisations jointes.

Fait à **Montpellier**..... le **21/04/2026**.....

pour faire valoir ce que de droit.

Signature :



Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Captures d'écran mockups desktop et mobile : Le Quai Antique (pages 44 à 48)
Diagramme de cas d'utilisation : Le Quai Antique (page 49)
Diagramme de sequence : Ju Coach Sportif (page 50)

ANNEXES

(Si le RC le prévoit)





RÉSERVATION

COUVERTS

DATE



HORAIRES

☐ 19:00	☐ 20:00
☐ 19:15	☐ 20:15
☐ 19:30	☐ 20:30
☐ 19:45	☐ 20:45

ALLERGIES

RESERVER



INSCRIPTION

NOM

PRÉNOM

E-MAIL

MOT DE PASSE

10 caractères min, 1 majuscule, 1 chiffre, 1 caractère spécial

CONFIRMEZ VOTRE MOT DE PASSE

ALLERGIES

Ex: gluten, lactose

Optionnel – sera pré-rempli lors de vos réservations

CONVIVES

S'INSCRIRE



accueil



RESERVER

LE QUAI ANTIQUE

ARNAUD MICHANT

DÉCOUVREZ NOTRE RESTAURANT,
UN HAVRE GASTRONOMIQUE
AU SEIN DE NOS PLUS BELLES MONTAGNES



MENTIONS LÉGALES



INSCRIPTION

NOM

PRÉNOM

E-MAIL

MOT DE PASSE

10 caractères min, 1 majuscule, 1 chiffre,
1 caractère spécial

CONFIRMEZ VOTRE MOT DE PASSE

ALLERGIES

CONVIVES

1	2	3	4	5
6	7	8	9	10

S'INSCRIRE



RÉSERVATION

COUVERTS

1	2	3	4	5
6	7	8	9	10



DATE



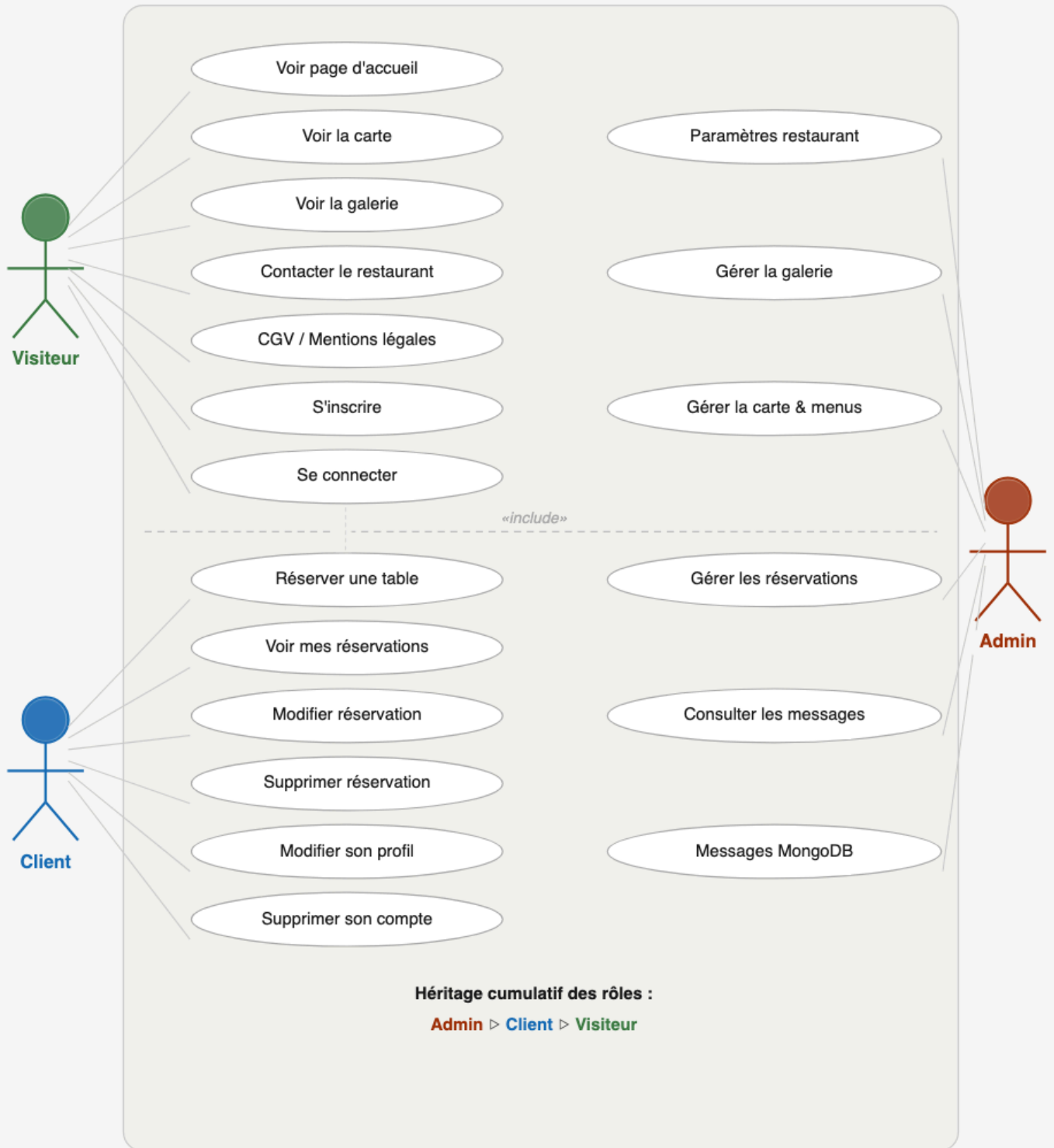
HORAIRES

☒ 19:00	☒ 20:00
☒ 19:15	☒ 20:15
☒ 19:30	☒ 20:30
☒ 19:45	☒ 20:45

ALLERGIES

RESERVER

DIAGRAMME DE CAS D'UTILISATION "QUAI ANTIQUE"



MCD Ju coach sportif

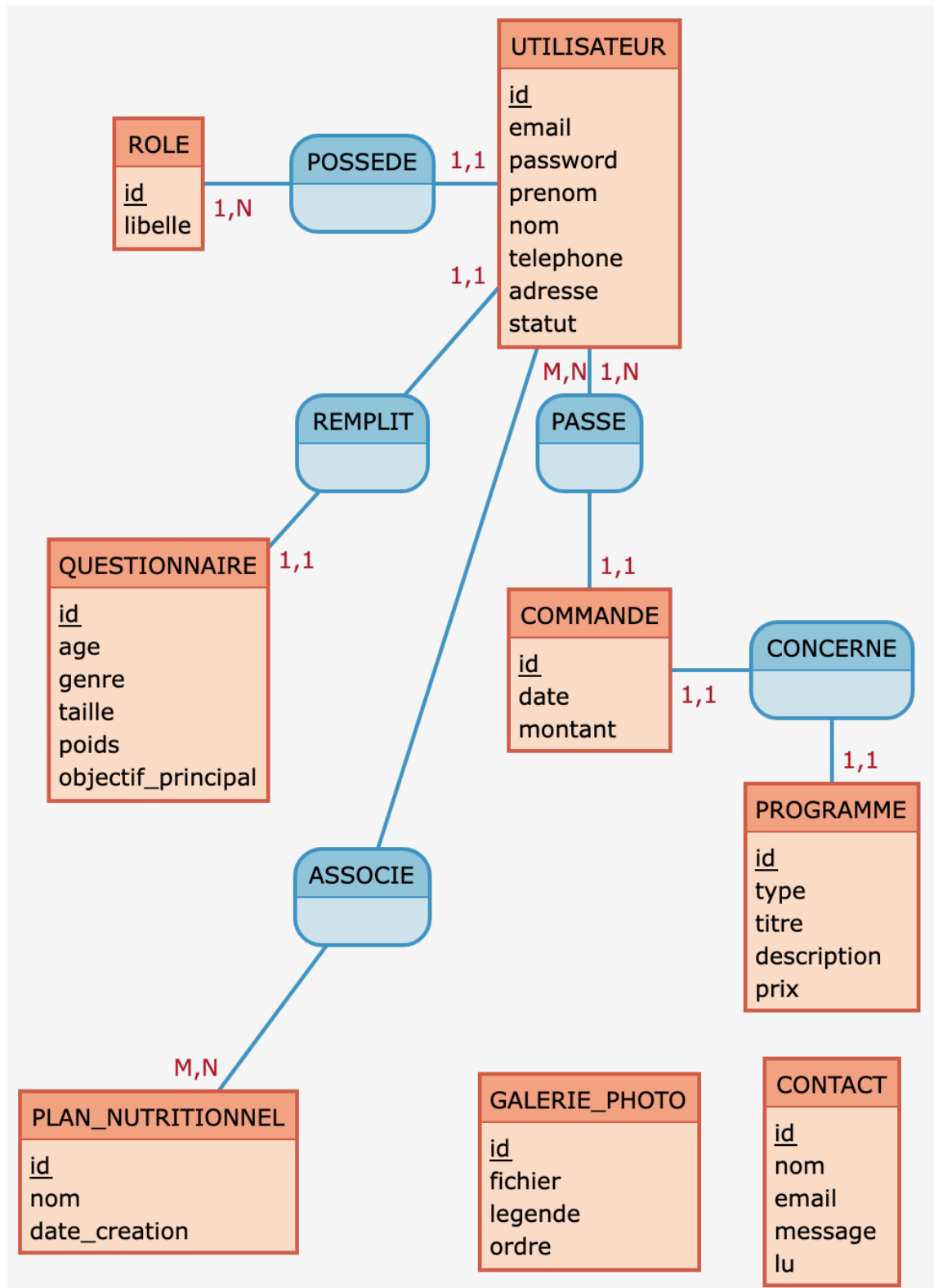


Diagramme de sequence Ju coach sportif

